

Міністерство освіти і науки України
Львівський національний університет імені Івана Франка
Факультет електроніки і комп'ютерних технологій

Звіт
про виконання лабораторної роботи №12
з курсу «Хмарні технології та великі дані»
на тему:
«Написати перевірку стану сервісів (Healthcheck)»

Виконав:
студент групи ФсІ-42
Трофимов Володимир
Перевірив:
ас. Керод Т. І.

Зміст

Тема.....	3
Мета	3
Вхідні дані	3
Завдання.....	4
Критерії виконання роботи.....	4
Теоретичні відомості	4
Структура програмної реалізації	5
Тестування програмної реалізації.....	5
Висновок	9
Код програми	10

Тема

Автоматизований контроль стану розподілених сервісів: конфігурація механізмів діагностики Healthcheck та синхронізація запуску контейнерів за критерієм готовності у Docker Compose.

Мета

Дослідження механізмів автоматичного моніторингу працездатності контейнеризованих процесів за допомогою директиви Healthcheck у специфікації Docker Compose. Проектування архітектури стійкого старту розподіленої системи шляхом заміни простого контролю запуску процесу на строгий критерій готовності залежних сервісів `condition: service_healthy`. Опанування методів написання діагностичних проб для баз даних і веб-сервісів, відстеження переходу контейнерів у статус `healthy` та аналіз поведінки оркестратора під час затримки ініціалізації середовища.

Вхідні дані

Параметр / Директива	Значення / Тип	Призначення
Специфікація конфігурації	<code>compose.yaml</code>	Маніфест опису взаємодії мульти-контейнерного додатку з розширеними блоками перевірки працездатності.
Сервіс <code>app</code>	<code>python:3.11-alpine (Flask)</code>	Веб-застосунок із реалізованим HTTP-маршрутом <code>/health</code> для верифікації внутрішньої готовності сервісу.
Сервіс <code>redis</code>	<code>redis:7-alpine</code>	In-memory база даних із системною перевіркою відгуку через штатну утиліту <code>redis-cli ping</code> .
Критерій залежності	<code>condition: service_healthy</code>	Правило черговості запуску, що блокує старт контейнера <code>app</code> до моменту успішного проходження проби сервісом <code>redis</code> .
Параметри проби (Healthcheck)	<code>interval, timeout, retries, start period</code>	Набір часових лімітів та лічильників спроб для визначення стабільного статусу контейнера.
Віртуальна мережа	<code>app-network (driver: bridge)</code>	Ізольований простір для внутрішньої взаємодії контейнерів та проходження діагностичних перевірок.

Завдання

- додати блок діагностики `healthcheck` до опису `service redis` у файлі `compose.yaml` із використанням штатної команди `redis-cli ping`.
- реалізувати в коді веб-сервісу `app.py` окремий маршрут `/health`, який перевіряє активність з'єднання з базою даних і повертає відповідний HTTP-статус.
- додати блок `healthcheck` для веб-сервісу `app`, що опитує маршрут `/health` та валідує готовність сервісу до обробки трафіку.
- оновити секцію `depends_on` сервісу `app`, замінивши базову залежність на розширену умову `condition: service_healthy`.
- здійснити запуск стека та зафіксувати часову затримку старту сервісу `app` до моменту підтвердження працездатності `redis`.
- перевірити фінальний статус контейнерів за допомогою команди `docker-compose ps`, підтвердивши наявність мітки (`healthy`).
- провести аудит системних логів та перевірити відповідь веб-ендпоінтів за допомогою утиліти `curl`.

Критерії виконання роботи

- *Об'єктивність діагностики:* статус `healthy` свідчить про реальну здатність компонента обробляти запити (для `Redis` — успішна відповідь `PONG`, для додатка — наявність робочого сокету та зв'язку зі сховищем).
- *Синхронізація запуску:* веб-застосунок `app` гарантовано не переходить до фази завантаження та виконання коду, доки контейнер `redis` не завершить власну внутрішню ініціалізацію і не отримає перший успішний статус здоров'я.
- *Стійкість статусів:* обидва сервіси після проходження тестових спроб переходять у стан `Up (healthy)` без циклічних перезапусків або виникнення статусу `unhealthy`.
- *Доступність моніторингу:* стан вузлів повністю прозорий та відображається у системних лістингах оркестратора і журналах контейнерів.

Теоретичні відомості

У стандартній моделі оркестрації `Docker` перевіряє працездатність контейнера виключно за фактом існування його головного процесу з `PID 1`. Проте ситуація, коли процес запущений, але перебуває у стані внутрішнього блокування (`deadlock`), збою виділення пам'яті або тривалої міграції схеми бази даних, залишається непомітною для базового моніторингу. У такому разі базовий статус `running` є хибним, оскільки застосунок фактично не здатний обробляти зовнішній трафік.

Механізм `Healthcheck` усуває цю проблему шляхом регулярного виконання всередині контейнера спеціалізованої діагностичної проби. Результат перевірки

визначається кодом завершення виконаної команди: 0 сигналізує про успішний стан (healthy), а 1 — про наявність збою (unhealthy). Робочий процес проби регламентується параметрами: interval (періодичність опитування), timeout (максимальний час очікування відповіді), retries (кількість невдалих спроб підряд перед оголошенням збою) та start_period (пільговий інтервал часу для ініціалізації процесу, коли невдалі проби не змінюють статус на аварійний).

Поєднання механізму Healthcheck із директивою depends_on: condition: service_healthy вирішує класичну проблему «холодного старту» розподілених систем (cold start race condition). Оркестратор призупиняє ініціалізацію залежних сервісів, запобігаючи падінню веб-застосунків через тимчасову недоступність сокетів баз даних або черг повідомлень.

Структура програмної реалізації

Архітектура стенду побудована на базі оновленого маніфесту compose.yaml та розширеного скрипта веб-сервера app.py.

Сценарій дослідження передбачає такі кроки:

- запуск мульти-контейнерного комплексу у фоновому режимі та безпосереднє спостереження за статусами контейнерів на етапі ініціалізації.
- фіксація проміжного стану (health: starting) для контейнера бази даних та очікування запуску залежного сервісу lab12-web-app.
- верифікація фінального стану системи, за якого обидва контейнери набувають постійного статусу Up (healthy).
- тестування працездатності сервісу через виклик діагностичного роуту /health та основного функціоналу / за допомогою утиліти curl.
- збір та аналіз зведених логів для підтвердження відсутності фатальних винятків на стадії підключення клієнта до сховища.
- завершення роботи та видалення робочого середовища викликом docker-compose down.

Тестування програмної реалізації

Етап 1. Валідація маніфесту та запуск сервісів із контролем черговості

Мета: Перевірка коректності синтаксису розширеного файлу compose.yaml та ініціалізація кластера з урахуванням умов готовності.

У консолі виконано перевірку конфігурації та її подальший запуск у фоновому режимі:

docker-compose config

docker-compose up -d

```

$ rdctl shell
/mnt/d/4_1/Cloud_Technologies/Lab_12 # docker-compose config
name: lab_12
services:
  app:
    command:
      - sh
      - -c
      - pip install --no-cache-dir flask redis && python app.py
    container_name: lab12-web-app
    depends_on:
      redis:
        condition: service_healthy
        required: true
    healthcheck:
      test:
        - CMD-SHELL
        - python -c "import urllib.request; urllib.request.urlopen('http://localhost:8080/health').read()"
      timeout: 3s
      interval: 5s
      retries: 3
      start_period: 5s
    image: python:3.11-alpine
    networks:
      app-network: null

/mnt/d/4_1/Cloud_Technologies/Lab_12 # docker-compose up -d
[+] up 3/3
✓ Network lab_12_app-network Created 0.1s
✓ Container lab12-redis Healthy 6.6s
✓ Container lab12-web-app Started 6.6s

```

Рис. 1 — Валідація маніфесту та запуск сервісів із контролем черговості

Аналіз результату: Маніфест успішно пройшов розбір парсером Docker Compose. Оркестратор запустив контейнер lab12-redis і тимчасово перевів створення сервісу lab12-web-app у режим очікування події успішного виконання початкової діагностичної проби Redis.

Етап 2. Фіксація переходів станів здоров'я контейнерів

Мета: Спостереження за динамікою зміни статусів контейнерів від початкової ініціалізації до переходу в стан повної готовності.

З інтервалом у кілька секунд виконано команди перевірки стану процесів:

docker-compose ps

```

/mnt/d/4_1/Cloud_Technologies/Lab_12 # docker-compose ps
NAME                IMAGE                PORTS                COMMAND                SERVICE    CREATED                STATUS
lab12-redis         redis:7-alpine       6379/tcp             "docker-entrypoint.s..." redis       About a minute ago    Up About a minute (healthy)
lab12-web-app       python:3.11-alpine   0.0.0.0:8080->8080/tcp, [::]:8080->8080/tcp "sh -c 'pip install ..." app        About a minute ago    Up About a minute (healthy)

```

Рис. 2 — Фіксація переходів станів здоров'я контейнерів

Аналіз результату: На перших секундах контейнер lab12-redis мав статус Up (health: starting). Лише після того, як вбудована утиліта redis-cli ping повернула код завершення 0, статус змінився на Up (healthy). Одразу після цієї події оркестратор ініціював старт контейнера lab12-web-app. Через 5 секунд веб-сервіс також успішно відповів на внутрішній запит /health, і в лістингу docker-compose ps обидва сервіси зафіксувалися зі статусом Up (healthy).

Етап 3. Функціональна перевірка діагностичного та робочого ендпоінтів

Мета: Верифікація коректності відповідей прикладного сервісу та підтвердження стабільної взаємодії із залежним сховищем даних.

Виконано звернення до сервісу за допомогою утиліти curl:

```
curl -i http://localhost:8080/health
```

```
curl http://localhost:8080
```

```
/mnt/d/4_1/Cloud_Technologies/Lab_12 # curl -i http://localhost:8080/health
HTTP/1.1 200 OK
Server: Werkzeug/3.1.9 Python/3.11.17
Date: Sat, 03 Oct 2026 16:12:45 GMT
Content-Type: application/json
Content-Length: 34
Connection: close

{"redis":true,"status":"healthy"}
/mnt/d/4_1/Cloud_Technologies/Lab_12 # curl http://localhost:8080
<h1>Docker Compose Healthcheck Lab</h1><p>Кількість запитів до Redis: <b>1</b></p>/mnt/d/4_1/Cloud_T
```

Рис. 3 — Функціональна перевірка діагностичного та робочого ендпоінтів

Аналіз результату: Службовий ендпоінт повертає HTTP-статус 200 OK та JSON-відповідь {"status":"healthy","redis":true}, що підтверджує активність внутрішнього з'єднання. Основний маршрут повертає HTML-сторінку з успішним збільшенням значення лічильника в пам'яті Redis.

Етап 4. Аналіз системних журналів та виведення системи з експлуатації

Мета: Аудит логів на предмет фіксації черговості запуску та завершення сесії тестування.

Викликано перегляд консольних потоків контейнерів:

```
docker-compose logs
```

```
docker-compose down
```

```
lab12-web-app | 127.0.0.1 - - [03/Oct/2026 16:14:27] "GET /health HTTP/1.1" 200 -
lab12-web-app | 127.0.0.1 - - [03/Oct/2026 16:14:32] "GET /health HTTP/1.1" 200 -
lab12-web-app | 127.0.0.1 - - [03/Oct/2026 16:14:38] "GET /health HTTP/1.1" 200 -
lab12-web-app | 127.0.0.1 - - [03/Oct/2026 16:14:43] "GET /health HTTP/1.1" 200 -
lab12-web-app | 127.0.0.1 - - [03/Oct/2026 16:14:48] "GET /health HTTP/1.1" 200 -
lab12-web-app | 127.0.0.1 - - [03/Oct/2026 16:14:53] "GET /health HTTP/1.1" 200 -
lab12-web-app | 127.0.0.1 - - [03/Oct/2026 16:14:59] "GET /health HTTP/1.1" 200 -
lab12-web-app | 127.0.0.1 - - [03/Oct/2026 16:15:04] "GET /health HTTP/1.1" 200 -
lab12-web-app | 172.18.0.1 - - [03/Oct/2026 16:15:05] "GET / HTTP/1.1" 200 -
lab12-web-app | 127.0.0.1 - - [03/Oct/2026 16:15:09] "GET /health HTTP/1.1" 200 -
lab12-web-app | 127.0.0.1 - - [03/Oct/2026 16:15:15] "GET /health HTTP/1.1" 200 -
lab12-web-app | 127.0.0.1 - - [03/Oct/2026 16:15:20] "GET /health HTTP/1.1" 200 -
lab12-web-app | 172.18.0.1 - - [03/Oct/2026 16:15:24] "GET /health HTTP/1.1" 200 -
lab12-web-app | 127.0.0.1 - - [03/Oct/2026 16:15:25] "GET /health HTTP/1.1" 200 -
```

Рис. 5 — Аналіз системних журналів

Аналіз результату: Журнали підтверджують повну відсутність збоїв типу ConnectionRefusedError, що зазвичай виникають у разі передчасного старту

клієнта до готовності бази даних. Виклик `docker-compose down` коректно зупинив виконання процесів та звільнив створену мережу.

Висновок

У ході виконання лабораторної роботи було досліджено та практично реалізовано механізми автоматичного контролю стану мікросервісних компонентів засобами директиви Healthcheck у Docker Compose.

Було спроєктовано багаторівневу систему перевірки життєздатності: для сховища Redis застосовано нативну перевірку готовності через команду `redis-cli ping`, а для веб-застосунку реалізовано спеціалізований ендпоінт `/health`, який валідує як власний стан веб-сервера, так і доступність залежного сервісу. Завдяки налаштуванню розширеної залежності `depends_on: condition: service_healthy` було усунуто проблему передчасного старту веб-сервісу, що забезпечило повністю безпомилкову ініціалізацію системи.

Практичні випробування за допомогою утиліт `docker-compose ps` та `curl` продемонстрували плавний перехід статусів від `starting` до стабільного `healthy` в обох контейнерах. Розроблена конфігурація підтвердила високу надійність оркестрації, гарантуючи стійкість сервісів до збоїв ініціалізації та автоматизацію діагностики розподіленої архітектури.

Код програми

Код до даної лабораторної роботи розміщений у репозиторії https://github.com/MrCatman137/Cloud_Technologies у директорії Lab_12 та має повну інструкцію запуску даної програми.